

%小车倒立摆动力系统建模

%小车质量 M , 单摆质量 m , 单摆重心到关节距离 l , 单摆转动惯量 I

%阻尼系数 b , 小车受力 F , 小车位置 x , 单摆与竖直向下方向角度 θ

% P 为单摆对关节作用力竖直方向分力, N 为水平方向分力

对小车水平方向受力分析有:

$$M\ddot{x} = F - N - b\dot{x}$$

对单摆水平方向受力分析有:

$$N = m \frac{d^2}{dt^2} (x + l \sin \theta)$$

也就是:

$$N = m\ddot{x} + ml\ddot{\theta} \cos \theta - ml\dot{\theta}^2 \sin \theta$$

对于单摆垂直方向受力分析有:

$$P = mg + m \frac{d^2}{dt^2} (l \cos \theta)$$

也就是:

$$P = mg - ml\ddot{\theta} \sin \theta - ml\dot{\theta}^2 \cos \theta$$

对于关节力矩有如下力矩平衡方程:

$$-Pl \sin \theta - Nl \cos \theta = I\ddot{\theta}$$

在工作点即 $\theta \approx 0$ 处局部线性化有:

$$\sin \theta \approx 0, \cos \theta \approx 1, \dot{\theta} \approx 0$$

一般情况下我们希望系统平衡点在 0 附近, 因此令:

$$\phi = \pi + \theta$$

则化简后有:

$$(I + ml^2)\ddot{\phi} - mgl\phi = ml\ddot{x}$$

$$(M + m)\ddot{x} + b\dot{x} - ml\ddot{\phi} = F = u$$

对上两式进行拉普拉斯变换有:

$$(I + ml^2)\Phi(s)s^2 - mgl\Phi(s) = mlX(s)s^2$$

$$(M + m)X(s)s^2 + bX(s)s - ml\Phi(s)s^2 = U(s)$$

则有单摆角度对输入的传递函数为:

$$G_{pend}(s) = \frac{\Phi(s)}{U(s)} = \frac{mls}{qs^3 + b(I + ml^2)s^2 - (M + m)mgl - bmg}$$

小车位置对输入的传递函数为:

$$G_{car}(s) = \frac{X(s)}{U(s)} = \frac{(I + ml^2)s^2 - mgl}{qs^4 + b(I + ml^2)s^3 - (M + m)mgl s^2 - bmgl s}$$

其中:

$$q = (M + m)(I + ml^2) - (ml)^2$$

得到传递函数之后输入 MATLAB 进行分析

```
%小车质量 0.5kg 单摆质量 0.2kg 阻尼系数 0.1
%单摆转动惯量 0.018 重力加速度 9.8 单摆长度 0.6m
m_car = 0.5;
m_pend = 0.2;
b = 0.1;
I = 0.018;
g = 9.8;
L = 0.3;

q = (m_car + m_pend)*(I + m_pend*L^2) - (m_pend*L)^2;

s = tf('s');
%输入(小车受力 F 或者 u)对小车位置传递函数
p_car = (((I + m_pend*L^2)/q)*s^2 - (m_pend*g*L/q))...
/(s^4 + (b*(I + m_pend*L^2))*s^3/q - ((m_car + m_pend)*m_pend*g*L)*s^2/q - b*m_pend*g*L*s/q);
%输入对单摆对地角度传递函数
p_pend = (m_pend*L*s/q)...
/(s^3 + (b*(I + m_pend*L^2))*s^2/q - ((m_car + m_pend)*m_pend*g*L)*s/q - b*m_pend*g*L/q);

sys_tf = [p_car; p_pend];
%设置输入为 u 也就是小车受力,输出为小车位置以及单摆角度
inputs = {'u'}; outputs = {'x'; 'phi'};
set(sys_tf, 'InputName', inputs, 'OutputName', outputs);
sys_tf
```

```
sys_tf =

From input "u" to output...
              1.68e-05 s^2 - 0.0002743
x:  -----
    1.008e-05 s^4 + 1.68e-06 s^3 - 0.000192 s^2 - 2.743e-05 s

              2.799e-05 s
phi: -----
    1.008e-05 s^3 + 1.68e-06 s^2 - 0.000192 s - 2.743e-05
```

Continuous-time transfer function.

```
%传递函数转换成状态空间描述
p = I*(m_car + m_pend) + m_car*m_pend*L^2;
A = [0          1          0          0;
     0  -(I+m_pend*L^2)*b/p  (m_pend^2*g*L^2)/p  0;
     0          0          0          1];
```

```

0 -(m_pend*L*b)/p m_pend*g*L*(m_car + m_pend)/p 0];
B = [0 (I + m_pend*L^2)/p 0 m_pend*L/p]';
C = [1 0 0 0; 0 0 1 0];
D = [0 0]';
%状态分别为:小车位置,小车加速度,单摆角度,单摆角速度
states = {'x' 'x_dot' 'phi' 'phi_dot'};
sys_ss = ss(A, B, C, D, 'statename', states, 'inputname', inputs, 'outputname', outputs)

```

```
sys_ss =
```

```

A =
           x      x_dot      phi  phi_dot
x           0         1         0         0
x_dot      0 -0.1667    1.633         0
phi        0         0         0         1
phi_dot    0 -0.2778    19.06         0

```

```

B =
           u
x           0
x_dot      1.667
phi         0
phi_dot    2.778

```

```

C =
           x      x_dot      phi  phi_dot
x           1         0         0         0
phi        0         0         1         0

```

```

D =
           u
x           0
phi         0

```

Continuous-time state-space model.

```

%查看系统极点以及系统矩阵 A 的特征值
%可以看出系统不稳定,有右半平面极点,A 的特征值有正实部
sys_poles = pole(sys_tf)

```

```

sys_poles = 7×1
           0
    4.3538
   -4.3776
   -0.1428
    4.3538
   -4.3776
   -0.1428

```

```
sys_eigs = eig(A)
```

```

sys_eigs = 4×1
           0
   -4.3776
   -0.1428
    4.3538

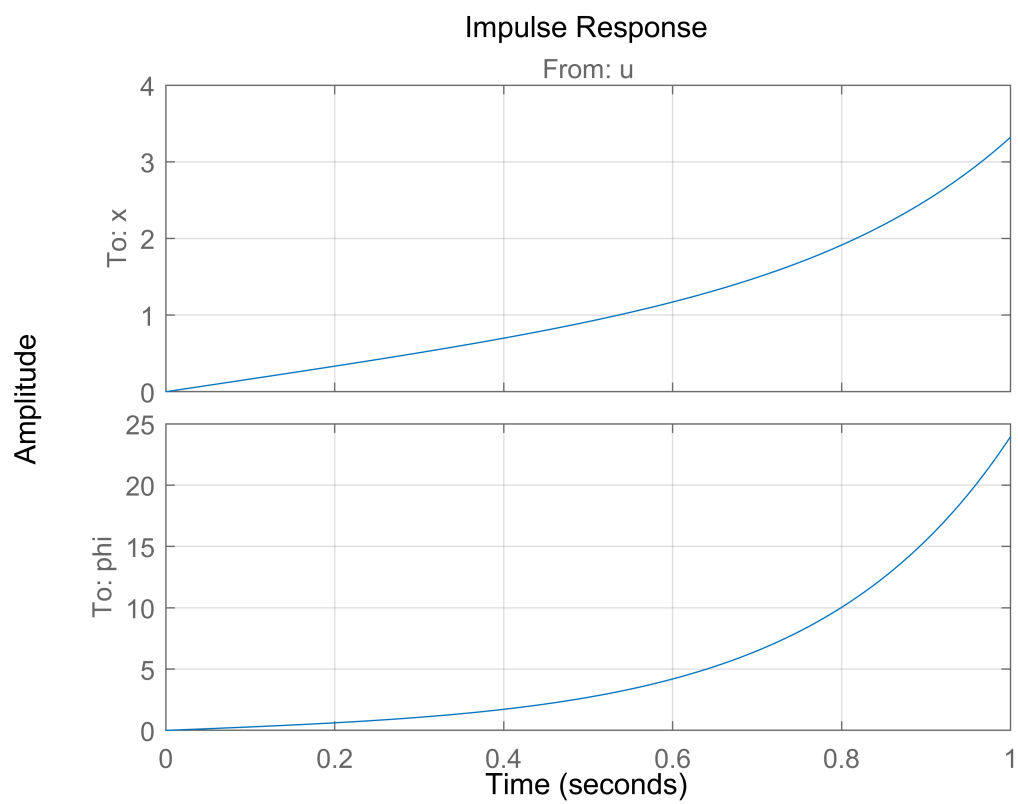
```

```

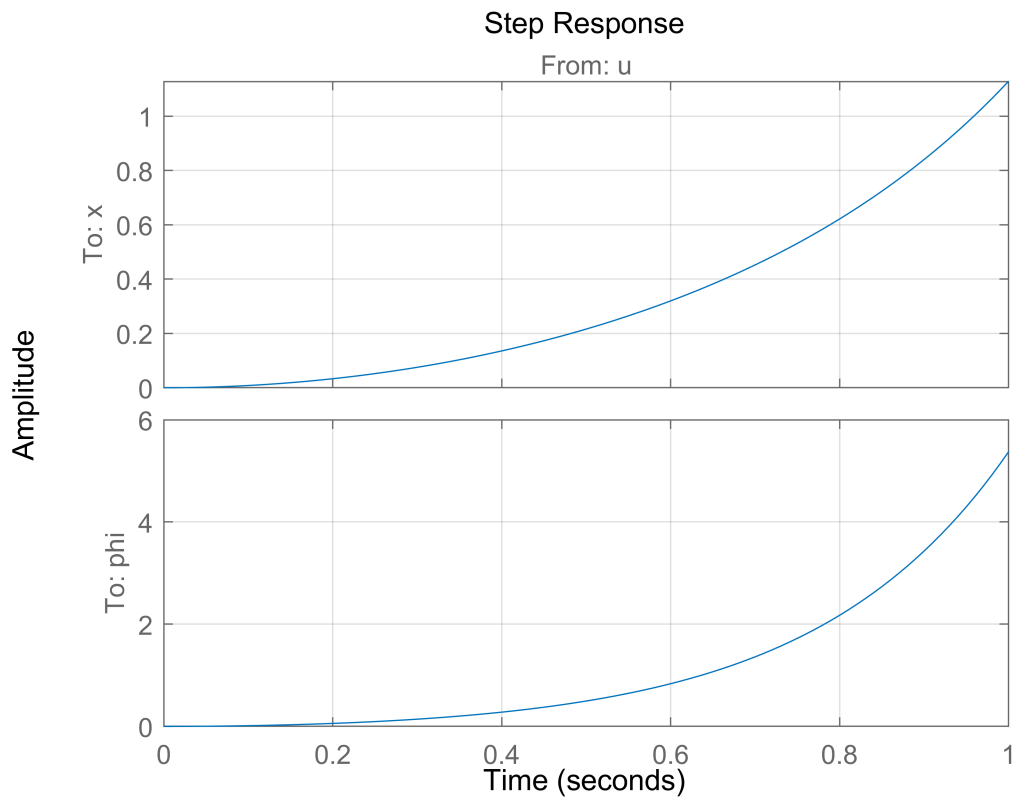
%查看开环系统单位脉冲响应,单位阶跃响应
%系统响应单调发散
impz(sys_tf, 0 : 0.01 : 1);

```

```
grid on;
```



```
step(sys_tf, 0 : 0.01 : 1);  
grid on;
```



%查看一下系统的能控性跟能观性
%可以看出系统是能控能观的, 可以进行状态反馈控制

```
cb = ctrb(A, B);
r_cb = rank(cb)
```

```
r_cb = 4
```

```
ob = obsv(A, C);
r_ob = rank(ob)
```

```
r_ob = 4
```

%设计全维状态观测器
%全维观测器极点配置为:-20 -15 -25 -10

```
foe_p = [-20 -15 -25 -10];
L = place(A', C', foe_p);
L = L'
```

```
L = 4x2
    34.6589    -4.9841
   279.9483   -84.8253
    -5.2688    35.1744
   -96.9965   309.6217
```

```
A_foe = A - L*C
```

```
A_foe = 4x4
   -34.6589    1.0000    4.9841    0
   -279.9483   -0.1667   86.4586    0
```

```

5.2688      0 -35.1744    1.0000
96.9965   -0.2778 -290.5662      0

```

%检查一下观测器极点是否确实配置到指定位置

```
Afoe_eigs = eig(A_foe)
```

```

Afoe_eigs = 4x1
-10.0000
-25.0000
-15.0000
-20.0000

```

%将 u 跟 y 合并成一个列向量 $[u; y]$, 则也需将 B 跟 L 按行合并成一个矩阵 $[B \ L]$

```
B_foe = [B L];
```

% C 为单位阵直接把状态向量输出

```
C_foe = eye(4);
```

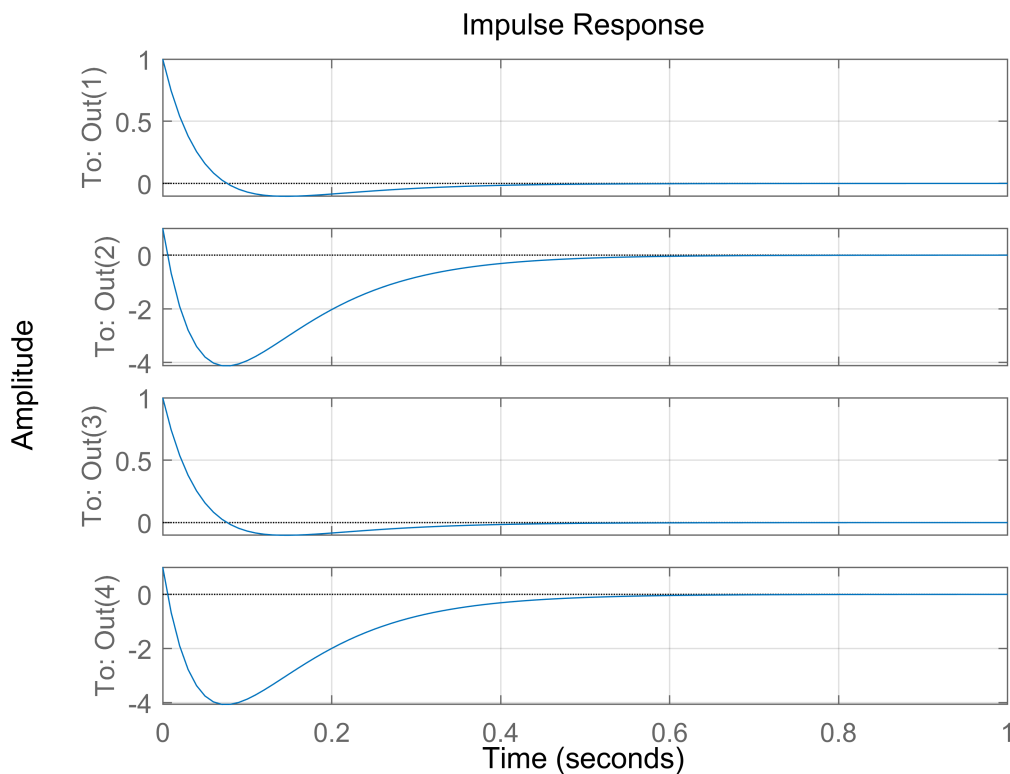
```
D_foe = zeros(4,3);
```

```
sys_foe = ss(A_foe, ones(4,1), C_foe, zeros(4, 1));
```

%查看观测器误差输出脉冲响应, 可以评估误差收敛速度, 若不满足要求可调整观测器极点配置

```
impz(sys_foe, 0 : 0.01 : 1);
```

```
grid on;
```



%设计降维观测器

%原系统中输出为位置 x 以及角度 ϕ , 因此选取 $R = [0 \ 1 \ 1 \ 0; 1 \ 0 \ 0 \ 1]$ 构成非奇异矩阵 P ;

%设计一个二维的观测器并结合 Q 即可将系统状态观测到

```
R = [0 1 1 0; 1 0 0 1];
```

%step1 构造非奇异矩阵 P

```
P = [C; R];
```

```

%step2 计算 P 逆并分块
Q = inv(P);
Q1 = Q( : , 1 : 2);
Q2 = Q( : , 3 : 4);
%step3 计算代数等价系统矩阵并分块
A_hat = P*A/P;
A11 = A_hat(1:2, 1:2);
A12 = A_hat(1:2, 3:4);
A21 = A_hat(3:4, 1:2);
A22 = A_hat(3:4, 3:4);
B_hat = P*B;
B1 = B_hat(1:2, : );
B2 = B_hat(3:4, : );
%step4 降维观测器极点配置为-15 -20,并综合出观测器反馈矩阵 L_hat
roe_p = [-15 -20];
L_hat = place(A22', A12', roe_p);
L_hat = L_hat';
%step5 计算观测器系统矩阵,输入矩阵,输出矩阵,前馈矩阵
A_roe = A22 - L_hat*A12;
%检查一下观测器极点是否确实配置到指定位置
A_roe_eigs = eig(A_roe)

```

```

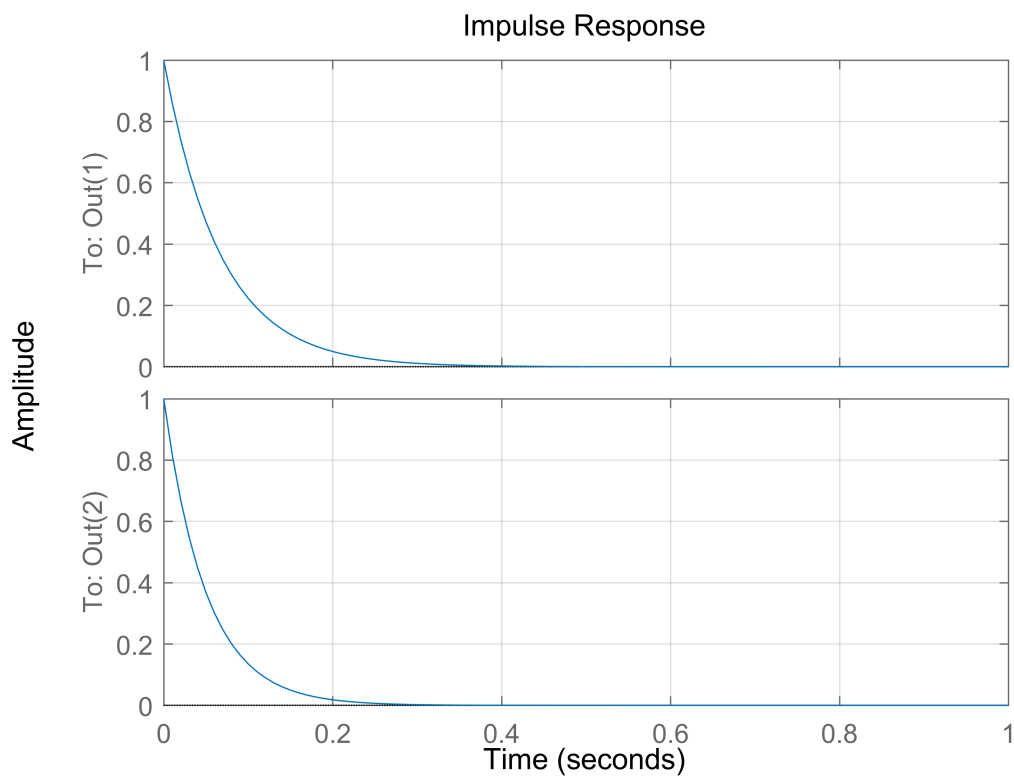
A_roe_eigs = 2×1
    -20
    -15

```

```

B_roe1 = A21 - L_hat*A11 + (A22 - L_hat*A12)*L_hat;
B_roe2 = B2 - L_hat*B1;
%此处将 u 与 y 合并成输入,因此将 B 也合并
B_roe = [B_roe2 B_roe1];
C_roe = eye(2);
D_roe = zeros(2, 3);
%定义 Q1 + Q2*L_hat 为最终状态输出中原输出 y 的系数 y_coef
y_coef = Q1 + Q2*L_hat;
%最终观测器状态方程为:z_dot = A_roo*z + B_roo*[u;y];
%最终状态输出方程为 x_hat = y_coef*y + Q2*z
sys_robv = ss(A_roe, ones(2, 1), eye(2), zeros(2, 1));
impulse(sys_robv, 0 : 0.01 : 1);
grid on;

```



%配置全状态反馈极点

p1 = -2 + 1i;

p2 = -2 - 1i;

p3 = -3 + 2i;

p4 = -3 - 2i;

%计算反馈矩阵 K

K = place(A, B, [p1 p2 p3 p4])

K = 1×4

-2.3878 -3.1122 23.4127 5.4073

Ac = A - B*K;

Bc = B;

Cc = C;

Dc = D;

inputs_cl = {'r'};

%闭环状态空间描述

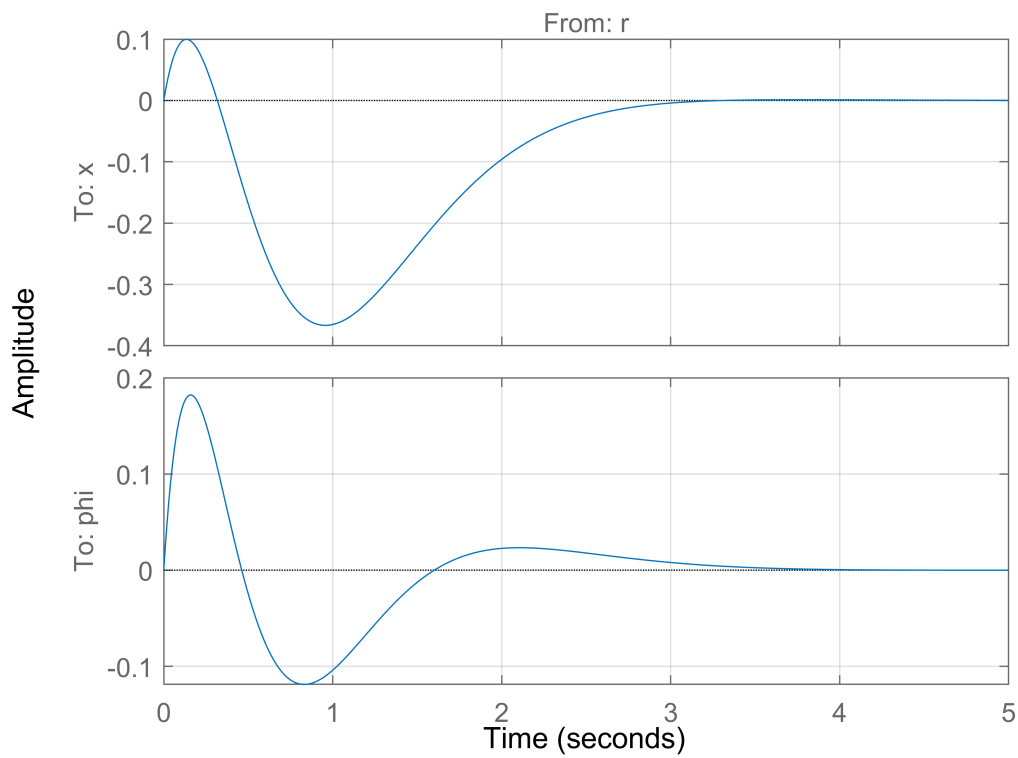
sys_cl = ss(Ac, Bc, Cc, Dc, 'StateName', states, 'InputName', inputs_cl, 'OutputName', outputs,

%查看闭环系统单位脉冲响应以及单位阶跃响应

impulse(sys_cl, 0 : 0.01 : 5);

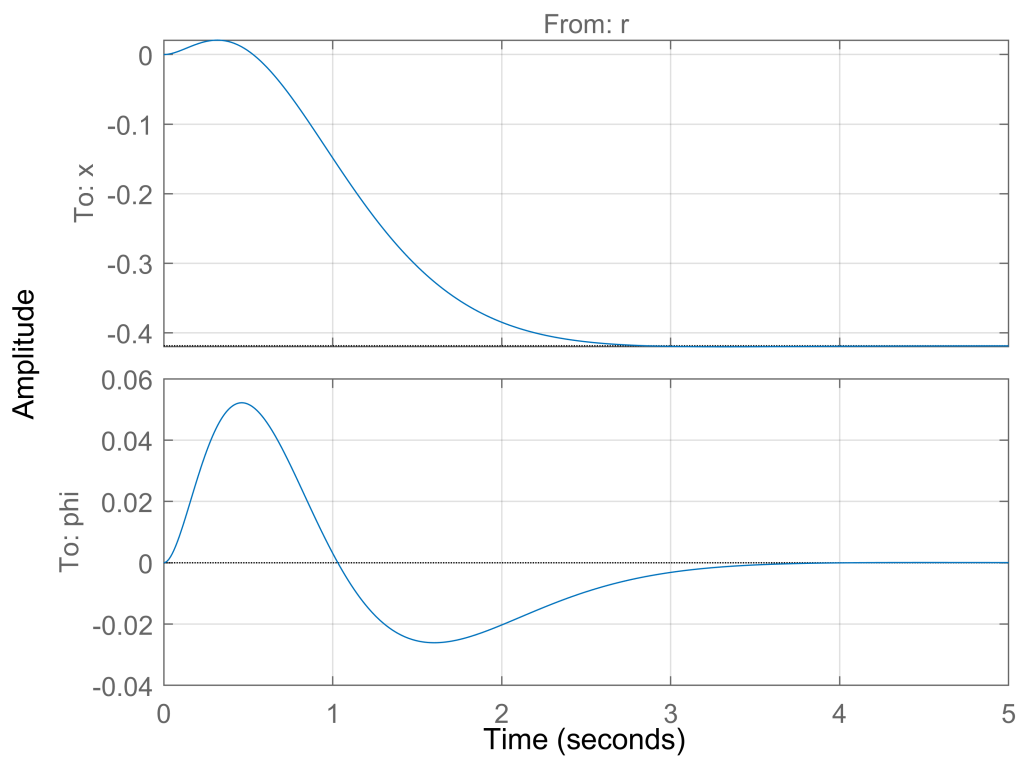
grid on;

Impulse Response



```
step(sys_cl, 0 : 0.01 : 5);
grid on;
```

Step Response



%使用 LQR 配置极点

```
Q = C'*C;
```

```
Q(1, 1) = 10;
```

%我们希望单摆角度尽快降到 0

```
Q(3, 3) = 1000;
```

%假定我们对输入不是很关心,即小车驱动电机输出力矩较大

```
R = 0.01;
```

```
K = lqr(A, B, Q, R);
```

```
Ac = A - B*K;
```

```
Bc = B;
```

```
Cc = C;
```

```
Dc = D;
```

```
sys_cl_lqr = ss(Ac, Bc, Cc, Dc, 'statename', states, 'inputname', inputs_cl, 'outputname', outputs_cl);
```

```
sys_cl_lqr =
```

```
A =
```

	x	x_dot	phi	phi_dot
x	0	1	0	0
x_dot	52.7	80.13	-606.8	-74.39
phi	0	0	0	1
phi_dot	87.84	133.6	-994.9	-124

```
B =
```

	r
x	0
x_dot	1.667
phi	0
phi_dot	2.778

```
C =
```

	x	x_dot	phi	phi_dot
x	1	0	0	0
phi	0	0	1	0

```
D =
```

	r
x	0
phi	0

Continuous-time state-space model.

%查看一下 lqr 综合出来的闭环极点

```
pole(sys_cl_lqr)
```

```
ans = 4x1 complex
```

```
-21.2038 +20.7507i
```

```
-21.2038 -20.7507i
```

```
-0.7199 + 0.6780i
```

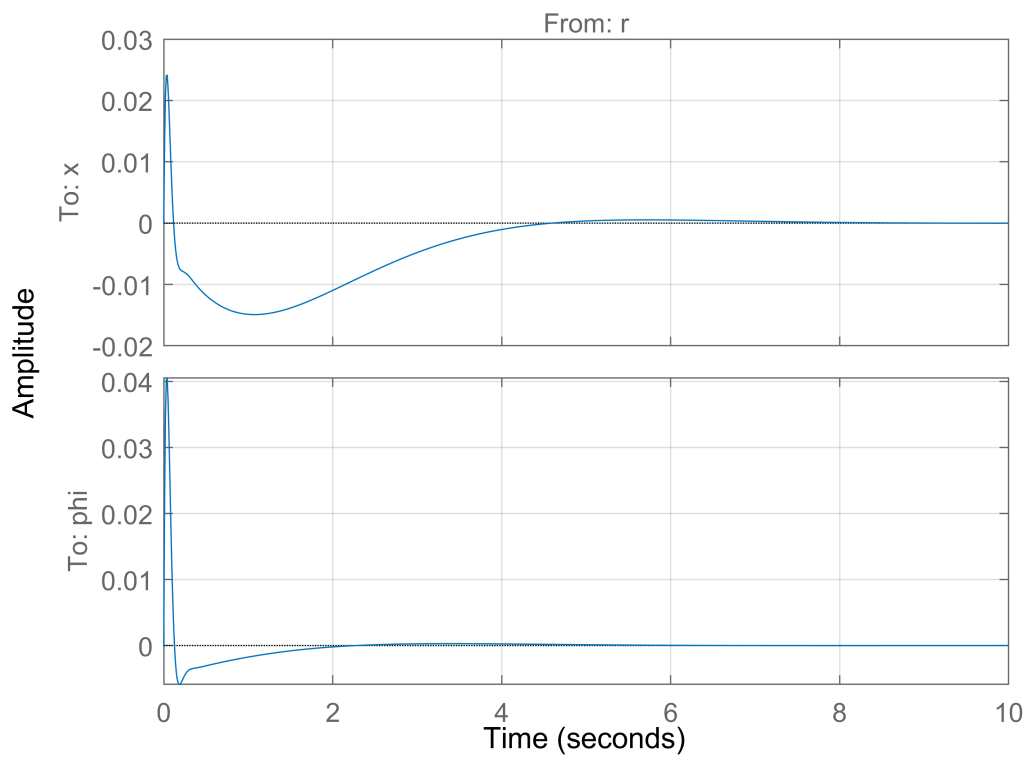
```
-0.7199 - 0.6780i
```

%查看使用 lqr 控制器的闭环系统单位脉冲响应以及单位阶跃响应

```
impz(sys_cl_lqr, 0 : 0.01 : 10);
```

```
grid on;
```

Impulse Response



```
step(sys_cl_lqr, 0 : 0.01 : 10);
grid on;
```

Step Response

